



UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3092 – Deep Learning y Sistemas Inteligentes

Laboratorio #5: Agentes en el Arcade Learning Environment (ALE): Space Invaders

Pedro Rubén Avila Cofiño, #23089

Guatemala, 13 de septiembre de 2026

1. El Arcade Learning Environment (ALE)

El **Arcade Learning Environment (ALE)** es un framework de investigación que expone más de 100 juegos originales de **Atari 2600** como entornos de aprendizaje por refuerzo con una interfaz programática uniforme (observación, acción, recompensa). Resuelve el problema de no contar con un banco estándar de entornos de percepción-y-control de dificultad realista: antes de ALE, evaluar agentes de RL en tareas con observaciones ricas (imágenes) y dinámicas complejas requería construir simuladores ad-hoc. ALE se apoya internamente en **Stella**, un emulador de código abierto del hardware original del Atari 2600 (el mismo binario `.bin` de la ROM corre sin modificaciones); ALE añade sobre Stella la extracción de frames, el cómputo de la recompensa a partir de la memoria del juego y una API tipo Gym para controlar el emulador paso a paso.

Gymnasium expone cada juego bajo el espacio de nombres `ALE/<Juego>-v5` (p. ej. `ALE/SpaceInvaders-v5`), que usa por defecto observaciones de imagen RGB e internamente fija un `frameskip` estocástico y una pequeña probabilidad de repetir la última acción (`repeat_action_probability=0.25`) para introducir estocasticidad y evitar que un agente memorice una secuencia fija de acciones. La variante con sufijo `-ram` (p. ej. `ALE/SpaceInvaders-ram-v5`) usa exactamente el mismo emulador y la misma dinámica, pero sustituye la observación de imagen por los 128 bytes de memoria de la consola. Otros parámetros de `gym.make` permiten ajustar este comportamiento: `frameskip` controla cuántos frames de emulación se ejecutan (y se repite la misma acción) por cada llamada a `step()`; `repeat_action_probability` controla la estocasticidad “sticky actions”; y `full_action_space` determina si se expone el conjunto completo de 18 acciones del joystick Atari o solo el subconjunto mínimo relevante para el juego (6 en Space Invaders).

El **frame skipping** es central en los entornos Atari porque el emulador corre internamente a 60 frames por segundo, una resolución temporal mucho más fina de la que necesita un agente de decisión: repetir cada acción durante k frames (típicamente $k=4$) reduce en ese mismo factor el número de decisiones -y de pasos de entrenamiento- necesarias para jugar un episodio completo, acelerando drásticamente tanto la simulación como el aprendizaje, sin perder capacidad de control real (las dinámicas relevantes del juego cambian poco en 4 frames = 1/15s).

Space Invaders es un juego de disparos de 1978 en el que el jugador controla un cañón en la parte inferior de la pantalla y debe destruir una formación de alienígenas que descienden progresivamente, evitando sus disparos y protegiéndose tras refugios destructibles; el juego termina cuando el cañón pierde sus 3 vidas o los alienígenas llegan al fondo de la pantalla. En ALE, el **reward** de cada paso corresponde directamente al incremento del marcador (*score*) del juego original en esos frames (p. ej. destruir un alienígena de la fila superior otorga más puntos que uno de la fila inferior), sin normalizar ni recortar por defecto.

2. Espacios de observación y acción en entornos Atari

Por defecto, `ALE/SpaceInvaders-v5` expone una observación `Box(0, 255, (210, 160, 3), uint8)`: un frame RGB completo de 210×160 píxeles. Esto contrasta fuertemente con `CartPole-v1` (visto en

el laboratorio anterior), cuya observación es un vector `Box(4,)` de bajo nivel (posición y velocidad del carro, ángulo y velocidad angular del poste) ya calculado por el simulador. Trabajar con imágenes en vez de vectores de bajo nivel implica que el agente debe aprender por sí mismo a extraer las características relevantes (posiciones, velocidades) a partir de píxeles crudos -un problema de percepción además de uno de control-, lo que multiplica la dimensionalidad de entrada ($210 \times 160 \times 3 = 100,800$ valores frente a 4) y hace casi obligatorio el uso de redes convolucionales y de más datos/tiempo de entrenamiento.

La variante `obs_type='ram'` expone directamente los 128 bytes de memoria de trabajo de la consola Atari 2600 (posiciones de sprites, marcador, vidas, etc., codificados por el propio juego). Es preferible sobre la imagen cuando se quiere reducir drásticamente el costo computacional y la dimensionalidad de entrada (128 valores en vez de $\sim 100K$) y se está dispuesto a perder interpretabilidad, ya que el significado de cada byte no está documentado y varía por juego.

El espacio de acción de Space Invaders es `Discrete(6)`, con significados `NOOP` (no hacer nada), `FIRE` (disparar), `RIGHT/LEFT` (mover el cañón) y `RIGHTFIRE/LEFTFIRE` (moverse y disparar en el mismo paso); este es el subconjunto *minimal* relevante para el juego (con `full_action_space=True` se expondrían las 18 acciones del joystick completo del Atari 2600).

El wrapper `AtariPreprocessing` de Gymnasium encapsula el preprocesamiento estándar usado en la literatura de DQN: convierte la imagen a escala de grises, la redimensiona a 84×84 , aplica *frame skipping* internamente (tomando el máximo entre los últimos dos frames para evitar parpadeo de sprites) y opcionalmente recorta la recompensa a $\{-1, 0, 1\}$. `FrameStackObservation` (antes `FrameStack`) apila los últimos n frames preprocesados en un solo tensor de observación; se usa junto con `AtariPreprocessing` porque un único frame estático no contiene información de movimiento (dirección/velocidad de sprites), algo indispensable para que una política pueda decidir bien.

3. Módulo de desarrollo: funciones para interactuar con ALE

Se implementó el módulo `scripts/ale_utils.py` con las cinco funciones reutilizables solicitadas, probadas end-to-end en el notebook `notebook/Lab05_ALE_SpaceInvaders.ipynb`:

- `crear_entorno(nombre_entorno, video_folder=None, episode_trigger=None, name_prefix, **kwargs)`: crea el entorno con `gym.make`; si se pasa `video_folder`, fuerza `render_mode='rgb_array'` y envuelve el entorno con `gymnasium.wrappers.RecordVideo`. Es genérica: funciona igual para `ALE/SpaceInvaders-v5` que para cualquier otro entorno de Gymnasium.
- `agente_aleatorio(observation, env)`: retorna `env.action_space.sample()`, usado como *baseline* sin entrenamiento.
- `agente_regla_simple(observation, env)`: política fija sin entrenamiento que alterna bloques de 10 pasos disparando y moviéndose a la derecha (`RIGHTFIRE`) y a la izquierda (`LEFTFIRE`); si el entorno no expone esas acciones (otro juego/entorno), cae de vuelta al muestreo aleatorio.
- `ejecutar_episodio(env, funcion_agente, max_steps=10000)`: corre un episodio hasta `terminated/truncated` o `max_steps`, retornando pasos y recompensa total acumulada.
- `generar_video_agente(nombre_entorno, funcion_agente, video_folder, name_prefix, n_episodios=1)`: combina las anteriores, graba `n_episodios` con `RecordVideo`, cierra el entorno (`env.close()`, indispensable para que el `.mp4` se escriba a disco) y retorna las rutas de video junto con las métricas de cada episodio.

Un detalle de infraestructura no trivial: el paquete `ale-py` de `nixpkgs` (usado en el `flake.nix` del proyecto) no incluye los binarios de ROM por política de empaquetado. El módulo fija la variable `ALE_ROMS_DIR` apuntando a `assets/roms/`, donde se distribuye `space_invaders.bin` (obtenido del paquete oficial `ale-py` de PyPI, cuyas ROMs están licenciadas para uso libre/investigación desde ALE 0.8.0, y verificado contra el hash MD5 que el propio `ale-py` espera), de modo que el entorno se pueda crear sin pasos manuales adicionales.

Resultados

Usando `ejecutar_episodio` directamente (sin grabación) se corrieron 5 episodios de cada agente sobre `ALE/SpaceInvaders-v5`:

Agente	Pasos promedio (5 eps)	Return promedio (5 eps)
agente_aleatorio	605.6	224.0
agente_regla_simple	374.2	75.0

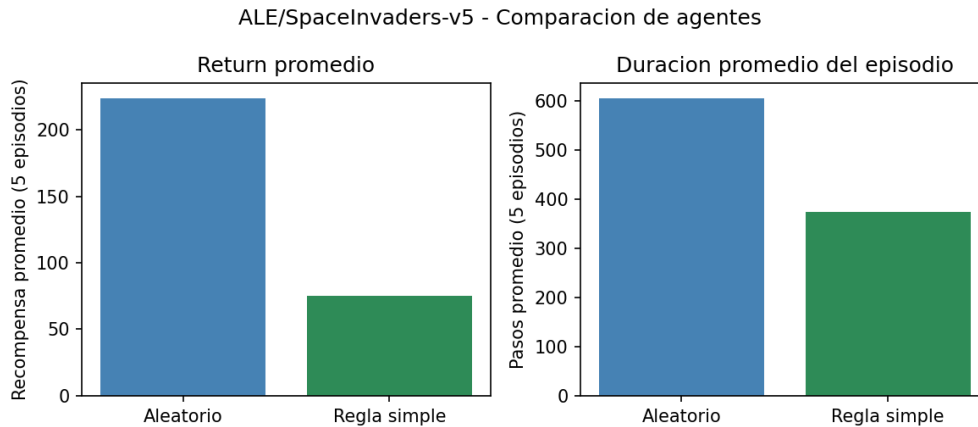


Figura 1: Return y duración promedio (5 episodios) del agente aleatorio vs. el agente de regla simple.

El agente aleatorio obtuvo, en promedio, mayor recompensa y episodios más largos que la regla fija de disparo/movimiento alterno; esto es consistente con que Space Invaders recompensa principalmente impactar alienígenas en posiciones variadas, algo que el muestreo aleatorio cubre mejor en pocos episodios que una regla determinista y repetitiva, la cual además puede quedar “atrapada” disparando siempre hacia el mismo lado de la formación.

Con `generar_video_agente` se generó el video entregable de 1 episodio completo del agente aleatorio (`entregables/videos/space-invaders-aleatorio-episode-0.mp4`: 373 pasos, return de 105.0), y adicionalmente uno del agente de regla simple (`space-invaders-regla-simple-episode-0.mp4`: 480 pasos, return de 155.0) a modo de comparación visual. Ambos videos, junto con las métricas impresas por episodio, quedan documentados en el notebook y en la carpeta `entregables/videos/`.